Project 2: System Architecture

EECS 581 Group 6

Introduction

This document serves as an addendum to EECS 581 Group 5's system_architecture.md document and a guide to the features added by EECS 581 Group 6 to the existing codebase.

The features added to the codebase by Group 6 can be summarized as follows:

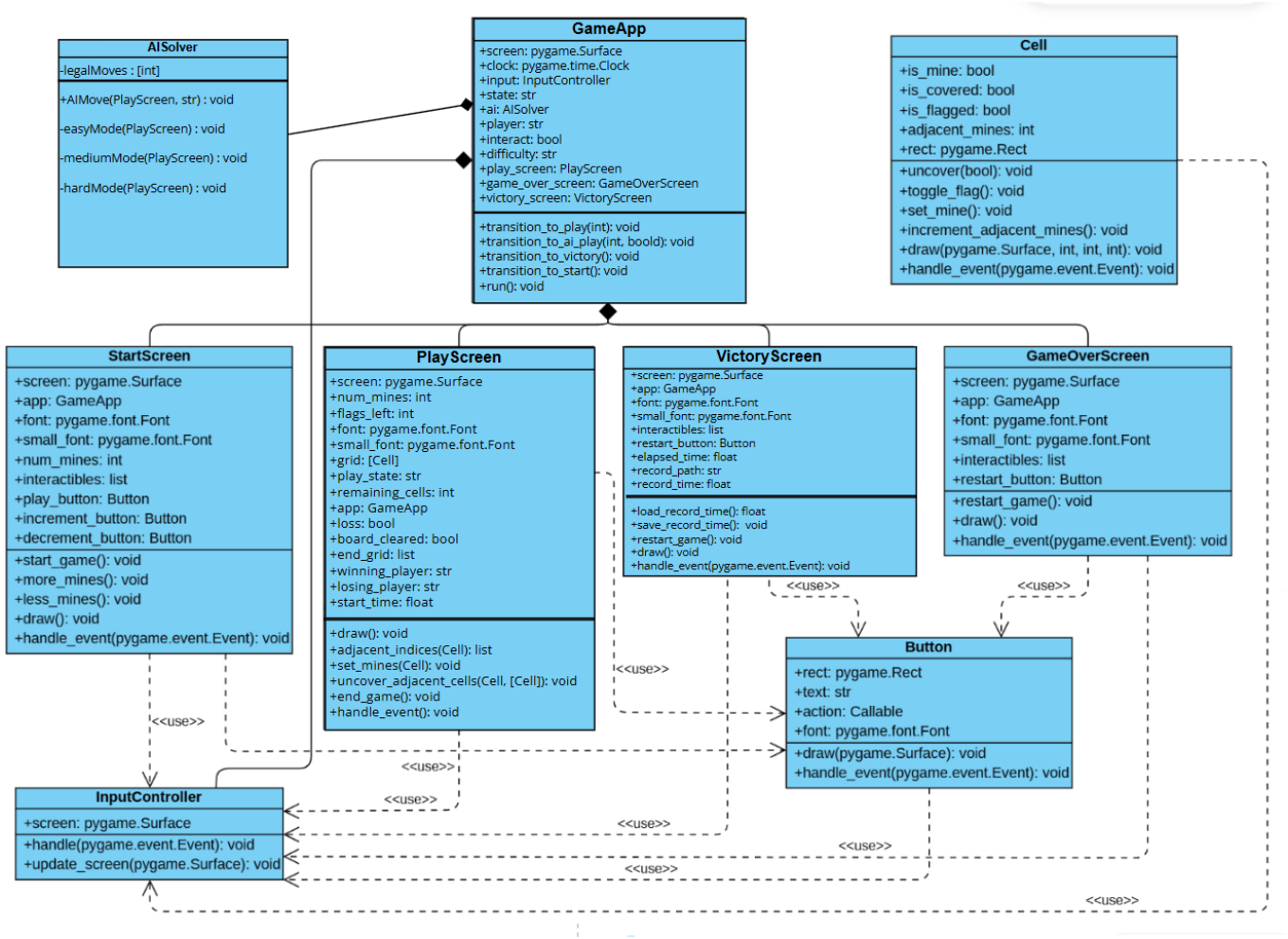
- An AI solver which boasts two modes, defined as:
  - **Automatic Mode**: The AI solver attempts a game itself, with no human interference.
  - **Interactive Mode**: The AI solver alternates with the player in a turn-based game, with the party uncovering the mine as the loser. The AI solver makes the first move.
- Three difficulty modes for the AI solver, defined as:
  - **Easy:** Cell selection is completely random.
  - **Medium:** Cell selection is random until the first cell with no adjacent mines is uncovered. The AI solver then makes strategic decisions for cell uncovering.
  - **Hard:** Cell selection is random, but cells with mines are excluded from the decision pool.
- Sound effects for uncovering cells, flagging cells, and uncovering mines.
- A timer feature, which displays the time to victory upon successful completion of a minesweeper game and stores the shortest time to be used as a high score.

This document will use the same headers as Group 5's system_architecture.md file, only highlighting the changes made and components added by Group 6. This document will also present an updated version of the Group 5's UML Class diagram, complete with the structures and changes needed for Group 6's additions.

System Architecture

Class Diagram

The updated UML class diagram for this project, complete with added and altered components, is displayed below.



Components

The components added or altered in this project are displayed below. Any components not listed here have not been changed and are described in Team 5's system_architecture.md document.

settings.py

- Description:
  - This file still stores values and constants used in other components.
- Changes:
  - Constants related to sound effects, such as ound effect files, loaded as pygame.mixer.Sound objects, were added.

GameApp

- Description:
    - Component which handles the main game loop and transitions between game screens and game states.
- Changes:
    - Added a new state, “ai play”, to the game loop, which permits the AI solver to make a move on the game grid. Interactive mode is implemented by switching between the “ai play” and “play” states.
    - Updated the transition to game victory by passing along the game time.

Cell

- Description:
    - Representation of a single cell in a minesweeper grid. Stores the properties of a cell(covered/uncovered, mine/no mine, flagged/not flaggeed) and the methods to change the properties of a single cell.
- Changes:
    - Added functionality to play the appropriate sound effect when a cell is flagged, clicked, or clicked to reveal a mine.

PlayScreen

- Description:
    - Main component that draws the game user interface, grid rendering, game logic, game move handling, and game state tracking.
- Changes:
    - Added attributes to track the winning or losing player, which is used for game state printing (“AI Wins”, “You lose!”, etc...).
    - Added functionality to store the start time of the game and calculate the elapsed time upon game victory.
    - Added functionality to support player alternation during interactive modes.
    - Corrected the faulty game-win condition, which required that all cells with mines be flagged, which was not in the initial Project 1 requirements.

StartScreen

- Description:
    - Component which renders the initial start screen for the game. Creates and handles all buttons for specifying mine count, AI difficulty, AI mode, and sound effect toggling.
- Changes:
    - Added buttons to specify AI modes (interactive and automatic). If the user does not press one, the program defaults to auto mode.
    - Added buttons to specify AI difficulty (easy, medium, hard).

        - **NOTE:** Once selected, a minesweeper will begin under the selected difficulty and current selected AI-mode.
    - Added button to toggle sound effects on and off.

VictoryScreen

- Description:
    - Component which renders the screen shown upon game victory. Displays the time to victory, and the shortest recorded time.
- Changes:
    - Added functionality to save and load a recorded time as a JSON file for saving and updating the high score (lowest time).

Sound Effect Files

- Description:
    - Three mp3 files for the “click”, “flag”, and “bomb” sound effects.

record.json

- Description:
    - Store the highest score(lowest game time) in a JSON file, to be loaded and updated by the VictoryScreen.

Data Flow

The addition of the AI solver generates a branch case in the data flow of the program, which is described below:

- User input (click) -> InputController validates input and directs it the active screen object’s handle_event() function.
- GameApp transitions between game states (“play”, “game_over”, “victory”, “start”, “ai play”) based on actions of the active screen.
    - In the “ai play” state, the AISolver will generate an input that feeds directly to the active screen. The InputController is bypassed.
- The active screen triggers the GameApp state transitions based on input received from the InputController or AISolver.
    - The PlayScreen updates the Board and Cell instances based on input.
- Changes to the Cell and Board states leads to changes in the UI displayed by the active screen.

Key Data Structures

This project uses the same key data structures as Group 5's Project 1, those being the Board, represented as a list of 100 Cell instances; Cell, an object representing a board cell; Screens, the PlayScreen, StartScreen, VictoryScreen, and GameOverScreen objects facilitating game UI rendering and event handling; GameApp, which manages transitions between game screens; and Button, used to register user input in various cases.

The timer and sound effect functionalities are added to existing data structures, as described above. The AISolver is implemented as a new class, as shown in the Components section and Class Diagram.

Assumptions

This project uses the same assumption, that a grid must only be 10x10 cells. Additionally, there can be only between 10-20 mines in a grid.